

Passi per riprodurre il buffer overflow

1. eseguire `make` per compilare le applicazioni
2. eseguire `python pattern.py 10 | ./buggy`: l'applicazione funziona
3. eseguire `python pattern.py 100 | ./buggy`: l'applicazione va in crash
4. lanciare `gdb ./buggy` per avviare l'applicazione in un debugger
5. impostare un breakpoint con `break main`
6. dare il comando `run < <(python3 pattern.py 100)`
7. `step instruction` fino al punto in cui viene allocato lo spazio per il buffer sullo stack: annotare l'indirizzo, quello sarà il nostro target provvisorio
8. `step instruction` fino a raggiungere l'istruzione `ret in main`
9. dare il comando `x/10c $rsp` per mostrare 10 caratteri sulla cima dello stack
10. prendere nota dei caratteri mostrati ed eseguire `python pattern.py CARATTERI` per trovare l'offset in byte
11. modificare `shell-it.py` per impostare l'indirizzo target ottenuto al punto 7
12. dare il comando `run < <(python shell-it.py OFFSET)` per lanciare l'applicazione.
13. eseguire in step fino all'istruzione `ret`
14. visualizzare la cima dello stack con `x/10x \rsp` per confermare che sia stato scritto l'indirizzo di ritorno corretto al buffer
15. eseguire `step instruction`: si noterà che non si saltà al codice dello shellcode: il compilatore ha allocato dello spazio per altre variabili locali d'appoggio, pertanto il target del punto 7 va corretto
16. utilizzare il comando `x/20x $rip` per mostrare i byte successivi alla prossima istruzione. Identificare le `nop` (`0x90`) e determinare l'indirizzo corretto del buffer. Prendere nota del delta tra il target provvisorio e quello corretto.
17. lanciando l'applicazione dentro gdb con `run < <(python shell-it.py OFFSET)` attiva effettivamente la shell. Fuori da GDB non funziona ancora (segfault): le variabili d'ambiente sono differenti.
18. lanciare `./buggy` fuori da GDB e, in un altro terminale, attaccarsi al processo in esecuzione con `sudo gdb --pid $(pidof buggy)`
19. `step up` fino alla funzione `main` per inserire un breakpoint all'istruzione `leave`.
20. inserire dell'input nel terminale dell'applicazione e, nel debugger, prendere nota dello stack pointer corrente: sarà il nostro nuovo indirizzo target provvisorio
21. applicare lo stesso offset annotato al punto 16: questo è il target finale da scrivere in `shell-it.py`
22. lanciando `python shell-it.py OFFSET | ./buggy` non si verificano crash ma l'applicazione termina. Questo è perché la shell avviata riceveva comandi da `stdin` (a causa del pipe `|`) ed ha consumato tutto l'output inserito (cioè niente, perché `./buggy` ha già letto l'input)
23. Lanciando l'applicazione come `(python shell-it.py OFFSET & cat) | ./buggy` si chiede di concatenare ciò che viene scritto da tastiera all'input iniziale
24. Non verrà mostrato alcun output, ma l'applicazione non termina: è diventata una shell interattiva